SUJAI RAJAN

Robotics Engineer

617-602-0833 send2sujai@gmail.com linkedin.com/in/sujai-r/ sujai-rajan.github.io/portfolio/

SUMMARY

Robotics Engineer with 4 years of experience turning ideas into reality, spanning perception, computer vision, sensor fusion, SLAM, autonomous navigation, and embedded integration. Building in ROS 2, Python, C++, and OpenCV, with mechanical and electrical work alongside the software. Owning systems from simulation through bring-up, debugging, and deployment into production.

PROFESSIONAL EXPERIENCE

Robotics Engineer | Futaba Corporation of America, Huntsville, AL **Mar 2025 - Present**

Sole engineer end-to-end on 3 production systems, 5 cells running across 2 lines

- Owned an epoxy dispensing cell end-to-end, concept through commissioning and operator training: a 6-DOF arm and two-part applicator built in-house to replace manual hand-gun application, lifting output from ~88 to ~235 boards/day and freeing a dedicated operator.
- Built the cell's control stack in Python (robot motion, digital I/O, threading, recipes, and the operator HMI) where each board variant loads as a runtime module without touching core code; collapsed seven blocking GPIO reads into one cached poll per cycle.
- Programmed per-variant dispense toolpaths from engineering drawings, controlling bead width by modulating robot speed against a fixed dispense rate, with a 3D-printed go/no-go nozzle gauge holding toolpaths valid across nozzle changes.
- Designed an inspection cell that checks every board before conformal coating in C and C++, verifying positional alignment, feature detection, and data-matrix identity at 100% traceability, with per-board worker threads pipelining motion and vision to inspect concurrently across a pallet, catching bad boards before a coating that can't be reworked.
- Raised coating throughput ~65% by replacing single-pallet hand loading with a two-stage conveyor feeding pallets continuously, on a 32-bit controller reflashed with custom Marlin firmware driving camera and conveyor motion over G-code with sensorless homing.
- Delivered two 6-DOF robotic vision cells (Python, ROS2, OpenCV) locating and decoding data-matrix codes on both board faces and linking them into one record at 100% traceability; applied multi-stage image preprocessing and optimized camera parameters for consistent, glare-free imaging under variable factory lighting, with operator HMI and image/CSV logging, and cut cycle time ~70% post-deployment.
- Integrated safety and I/O across cells (E-Stop, light curtains, photoelectric sensors, tower lights) with automatic pause/resume, fault recovery, and rev-controlled work instructions so operators run independently on day one.
- Validated each cell offline against golden and failed units before release, with startup self-tests for I/O, robot nodes and camera connectivity, and axis homing, plus failed-image logging for debugging; shipped as compiled executables, one versioned build per release.

Robotics Software Engineer, Contract | Tekflaire, TX **Aug 2024 - Mar 2025**

- Architected and delivered an outdoor autonomous mobile robot stack on client hardware (2D LiDAR, 9-axis VectorNav IMU, GPS, and camera on Jetson Nano), ROS 2 sensor bring-up, TF and time synchronization, and field validation through client handoff.
- Cut heading error from 10-30 degrees to ~2 degrees through hard/soft-iron magnetometer calibration and EKF fusion of IMU, LiDAR, and GPS, then tuned Nav2 costmaps, DWA, and A* against logged hardware runs to close the sim-to-real gap.
- Implemented the perception layer in OpenCV (terrain segmentation and feature detection classifying traversable ground from a single camera under variable outdoor lighting), adapting published traversability methods to run at ~10 fps on Jetson Nano.

Robotics Engineer | Citus Infotech, India **Aug 2020 - Jul 2022**

- Mapped warehouse floors using 2D LiDAR SLAM in ROS with Cartographer, correcting wheel-odometry yaw drift through IMU fusion and relying on pose-graph loop closure to hold map consistency across large floor loops.
- Deployed autonomous navigation using a pre-mapped occupancy grid with move_base and AMCL, tuning particle counts, costmap inflation radius, and DWA sim_time to hold reliable clearance through narrow warehouse aisles.
- Configured motion-control boards and firmware for a conveyor-driven sorting cell as part of a cross-functional engineering team, implementing PID velocity control in C++ and tuning loop response to eliminate oscillation under variable load.

SKILLS

Languages & Frameworks: Python, C++, C, MATLAB, Linux, Git, ROS, ROS2, Gazebo, RViz, MoveIt2, Nav2, AMCL, Cartographer
Navigation & Estimation: SLAM, localization, mapping, sensor fusion, EKF/Kalman filtering, costmaps, DWA, A*, motion planning, sim-to-real
Perception & Computer Vision: OpenCV, image processing, camera calibration, pose estimation, object detection, stereo/3D vision, GTSAM
Machine Learning: PyTorch, TensorFlow, Keras, CNNs, transfer learning, supervised / unsupervised / reinforcement learning
Hardware & Embedded: LiDAR, IMU, GPS, camera, Jetson Nano, Raspberry Pi, ESP32, Marlin firmware, G-code, motion control, GPIO
Manufacturing & Systems: 6-DOF industrial arms, HMI, MES, SMEMA, machine vision, traceability, CAD (SolidWorks, Inventor, AutoCAD)

EDUCATION

Master of Science in Robotics | Northeastern University, Boston, MA **Sep 2022 - May 2024**
Key Coursework: Autonomous Field Robotics, Pattern Recognition & Computer Vision, Machine Learning, Control Systems, Neural Networks & Deep Learning, Robot Mechanics & Control, Robotic Science & Systems, Robot Sensing & Navigation
Bachelor of Technology in Mechanical Engineering | Vellore Institute of Technology, India **Aug 2017 - Apr 2021**

PROJECTS

Household Automation Robot on Hello Robot Stretch RE2 (ROS2, Python, Gazebo, RViz, MoveIt2, SLAM, OpenCV, 2D LiDAR, ArUco): Took a household assistant from Gazebo simulation onto the physical Stretch RE2, covering mapping, navigation, object and face detection, pose estimation, grasping, and delivery, and completing 13 of 15 live delivery runs with ArUco pose estimation and gesture interaction.

3D Point Cloud Reconstruction using SfM (Python, OpenCV, GTSAM, Open3D, SIFT, RANSAC): Reconstructed 3D point clouds from sequential images with SIFT features, fundamental matrix estimation, triangulation, RANSAC outlier rejection, and GTSAM bundle adjustment, reducing reprojection error by 93%.

Underwater Image Stitching and Alignment Optimization (Python, OpenCV, GTSAM, SIFT, RANSAC, Levenberg-Marquardt): Mosaicked underwater AUV imagery into large-scale maps, preprocessing low-contrast frames with CLAHE and aligning through SIFT/RANSAC; optimized homographies via GTSAM factor-graph optimization, reducing alignment error by ~78%.

GPS and IMU Fusion for Navigation (ROS, Linux, Python, Kalman Filter, Sensor Fusion, GPS, IMU): Fused GPS and IMU topics through a Python Kalman filter to improve 2D pose estimation for autonomous navigation, reducing IMU drift through bias characterization.

Enhancing Quadcopter Stability using MPC, LQR, and Robust Control (MATLAB, Simulink, Python, MPC, LQR, mu-Synthesis, Monte Carlo): Implemented MPC for trajectory tracking and LQR to balance tracking against energy consumption; held stability under parametric uncertainty using mu-synthesis robust control validated through Monte Carlo simulation.

Autonomous Car Simulation on Udacity Self-Driving Car Simulator (Python, Keras, TensorFlow, OpenCV, imgaug, Unity3D): Trained an end-to-end CNN steering model on 10k+ augmented frames using YUV normalization, dropout, and behavior cloning; ran real-time inference with PID feedback in Udacity's simulator, holding <0.1 rad error across multi-track tests.

Augmented Reality Tool (C++, OpenCV, Camera Calibration, Pose Estimation, ORB): Calibrated a camera to 0.14 px reprojection error, projected virtual objects into the chessboard frame on live video, and extended the pipeline to ORB keypoint detection.

Gesture Recognition System using CNNs (C++, Keras, TensorFlow, OpenCV, CNN): Classified 18 hand gestures at 97%+ test accuracy with a multi-layer CNN, segmenting hands with OpenCV background subtraction, motion detection, and contours, robust to lighting.

Real-time 2-D Object Recognition (Python, OpenCV, Hu Moments, kNN, Morphological Filtering): Segmented with thresholding and morphological filtering, extracted invariant Hu Moments from connected components, and classified with kNN scored by confusion matrix.

Content-Based Image Retrieval (Python, OpenCV, Histograms, Sobel, Law Filters): Retrieved images on combined color, texture, and spatial cues, beating SSD and histogram-intersection baselines with weighted spatial histograms and Sobel and Law-filter texture features.

Recognition using Deep Networks (Python, PyTorch, Torchvision, CNN, Transfer Learning): Trained a CNN for digit recognition, then transfer-learned it to new symbol sets including Greek alphabets, tuning through layer, filter, dropout, and hidden-node sweeps.

Human Activity Recognition using MPM and 1D CNN (Python, Keras, 1D CNN, UCI Dataset, Adam, MaxPooling): Classified 8 activity classes from ambient sensor data at ~91% test accuracy using a 5-layer network with ReLU and MaxPooling, after collinear-feature removal, input scaling, and upsampling to balance classes.